

✓ Importar Módulos

```
import pandas as pd
import requests
import zipfile
import os
import geopandas
import folium
from IPython.display import display
```

✓ Descargar Datos

Los datos de viajes se pueden descargar desde una URL de la forma:

url = "https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-03.parquet"

Estos datos se complementan con los datos que identifican a cada zona de inicio y fin de viaje.

```
zones_url = "https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv"
```

Tambien se cuenta con datos de la ubicación geográfica de cada zona

```
geo_url = "https://d37ci6vzurychx.cloudfront.net/misc/taxi_zones.zip"
```

✓ Otros Set de Datos de viajes

La ciudad de NY pone a disposición datos de 4 tipos de flota de vehículos. Si se desea cambiar entre tipos de vehículos se puede cambiar el prefijo para generar la url de descarga de los otros tipos de vehículos.

Tipo de datos	Prefijo
Yellow Taxi	<code>yellow_tripdata</code>
Green Taxi	<code>green_tripdata</code>
For-Hire Vehicle	<code>fhv_tripdata</code>
High Volume FHV	<code>fhvhv_tripdata</code>

✓ Generación automática de URL de datos

Se define una función que genera automáticamente la URL para descargar los datos, según el tipo de vehiculo, año y mes.

```
def construir_url_tlc(tipo, anio, mes):  
    """  
    tipo puede ser:  
    - 'yellow'  
    - 'green'  
    - 'fhv'  
    - 'fhvhv'  
    """  
  
    prefijos = {  
        "yellow": "yellow_tripdata",  
        "green": "green_tripdata",  
        "fhv": "fhv_tripdata",  
        "fhvhv": "fhvhv_tripdata"  
    }  
  
    prefijo = prefijos[tipo]
```

```
mes = str(mes).zfill(2)

return f"https://d37ci6vzurychx.cloudfront.net/trip-data/{prefijo}_{anio}-{mes}.parquet"
```

Se puede descargar primero el archivo en local y luego importarlo con el path local, o directamente importarlo desde la url.

```
trips_url = construir_url_tlc("yellow", 2024, 1)
print(f'La url es: {trips_url}')
```

La url es: https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-01.parquet

✓ Importar Datos

Datos de los viajes

```
df_viajes = pd.read_parquet(trips_url)

print(df_viajes.shape)
print(df_viajes.head())
```

```
(2964624, 19)
  VendorID tpep_pickup_datetime tpep_dropoff_datetime passenger_count \
0         2  2024-01-01 00:57:55  2024-01-01 01:17:43             1.0
1         1  2024-01-01 00:03:00  2024-01-01 00:09:36             1.0
2         1  2024-01-01 00:17:06  2024-01-01 00:35:01             1.0
3         1  2024-01-01 00:36:38  2024-01-01 00:44:56             1.0
4         1  2024-01-01 00:46:51  2024-01-01 00:52:57             1.0

  trip_distance  RatecodeID store_and_fwd_flag  PULocationID  DOLocationID \
0             1.72         1.0                N           186             79
1             1.80         1.0                N           140            236
2             4.70         1.0                N           236             79
```

3	1.40	1.0		N	79	211
4	0.80	1.0		N	211	148

	payment_type	fare_amount	extra	mta_tax	tip_amount	tolls_amount	\
0	2	17.7	1.0	0.5	0.00	0.0	
1	1	10.0	3.5	0.5	3.75	0.0	
2	1	23.3	3.5	0.5	3.00	0.0	
3	1	10.0	3.5	0.5	2.00	0.0	
4	1	7.9	3.5	0.5	3.20	0.0	

	improvement_surcharge	total_amount	congestion_surcharge	Airport_fee
0	1.0	22.70	2.5	0.0
1	1.0	18.75	2.5	0.0
2	1.0	31.30	2.5	0.0
3	1.0	17.00	2.5	0.0
4	1.0	16.10	2.5	0.0

Nombres de las localizaciones

```
df_zones = pd.read_csv(zones_url)

print(df_zones.head())
```

	LocationID	Borough	Zone	service_zone
0	1	EWB	Newark Airport	EWB
1	2	Queens	Jamaica Bay	Boro Zone
2	3	Bronx	Allerton/Pelham Gardens	Boro Zone
3	4	Manhattan	Alphabet City	Yellow Zone
4	5	Staten Island	Arden Heights	Boro Zone

Datos de localización geográfica

```
# 1. Descargar y extraer (solo si no existe para ahorrar tiempo)
local_zip = "taxi_zones.zip"
extract_dir = "taxi_zones_data"
```

```
if not os.path.exists(local_zip):
    response = requests.get(geo_url)
    with open(local_zip, 'wb') as f:
        f.write(response.content)

with zipfile.ZipFile(local_zip, 'r') as zip_ref:
    zip_ref.extractall(extract_dir)

shp_path = None
for root, dirs, files in os.walk(extract_dir):
    for file in files:
        if file.endswith(".shp"):
            shp_path = os.path.join(root, file)
            break

if shp_path:
    geo_df = geopandas.read_file(shp_path)
    if geo_df.crs != "EPSG:4326":
        geo_df = geo_df.to_crs(epsg=4326)

    bounds = geo_df.total_bounds
    center = [(bounds[1] + bounds[3]) / 2, (bounds[0] + bounds[2]) / 2]

    # Cambiamos a 'CartoDB positron' que tiene políticas de referer menos estrictas en Colab
    m = folium.Map(location=center, zoom_start=10, tiles='CartoDB positron')

    folium.GeoJson(
        geo_df,
        name="Taxi Zones",
        style_function=lambda x: {
            'fillColor': '#3186cc',
            'color': 'black',
            'weight': 0.5,
            'fillOpacity': 0.4
        }
    ).add_to(m)
```

```
        folium.LayerControl().add_to(m)
    else:
        print("No se encontró el archivo .shp.")
```

```
# 2. Update map with correct LocationID field for tooltips
m_updated = folium.Map(location=center, zoom_start=10, tiles='CartoDB positron')

folium.GeoJson(
    geo_df,
    name="Taxi Zones",
    style_function=lambda x: {
        'fillColor': '#3186cc',
        'color': 'black',
        'weight': 0.5,
        'fillOpacity': 0.4
    },
    tooltip=folium.GeoJsonTooltip(fields=['LocationID'], aliases=['LocationID:'])
).add_to(m_updated)

folium.LayerControl().add_to(m_updated)
display(m_updated)

# 3. Table with geographic coordinates of each zone's center
# We project to a metric system (EPSG:3857) to calculate accurate centroids, then back to geographic
df_centroids = geo_df.copy()
df_centroids['centroid'] = geo_df.to_crs(epsg=3857).geometry.centroid.to_crs(epsg=4326)
df_centroids['longitude'] = df_centroids['centroid'].x
df_centroids['latitude'] = df_centroids['centroid'].y

# Selecting relevant columns for the table (using the correct LocationID column)
tabla_centros = df_centroids[['LocationID', 'zone', 'longitude', 'latitude']]
print("\nTabla de coordenadas de los centros de cada zona:")
display(tabla_centros.head())
```



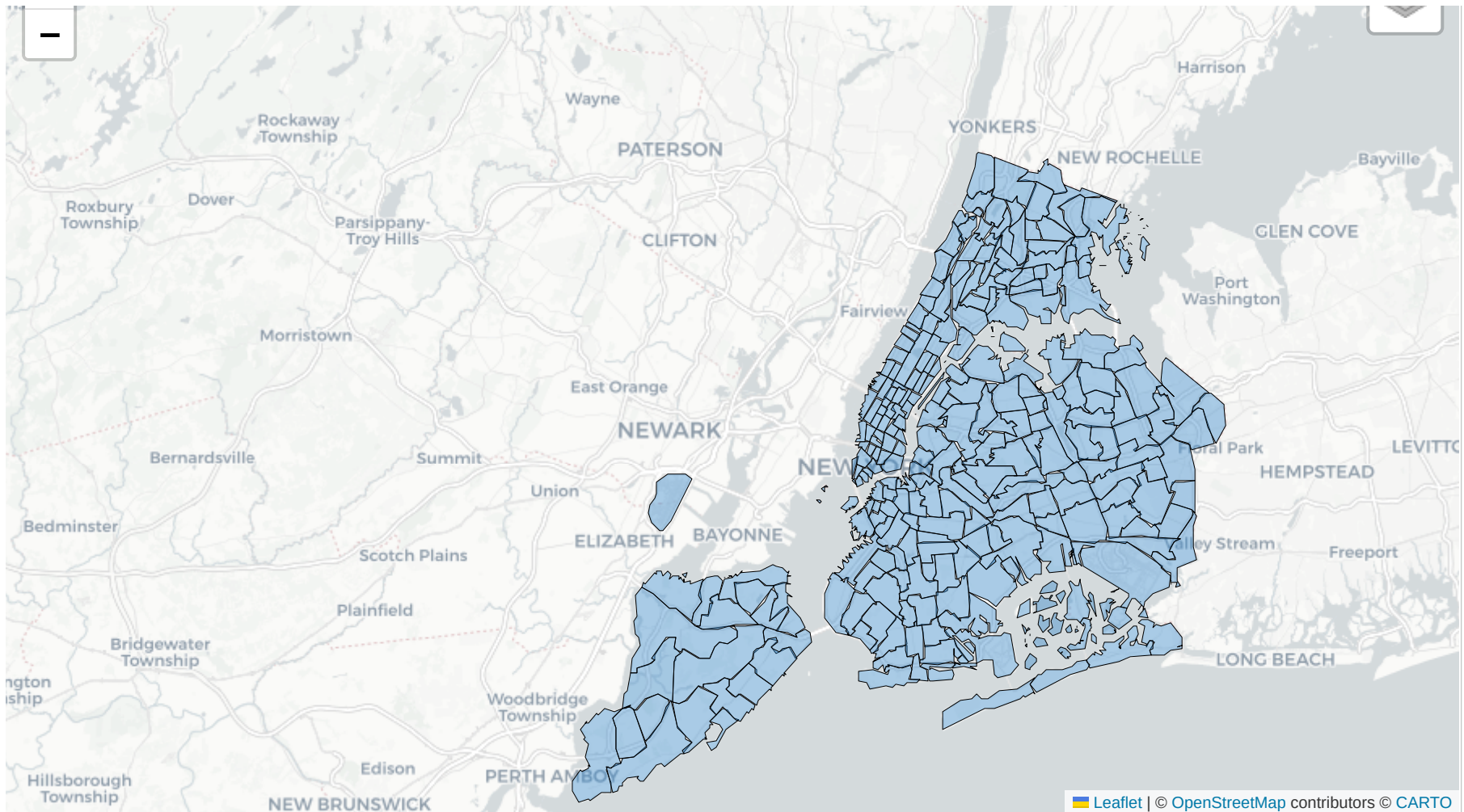


Tabla de coordenadas de los centros de cada zona:

	LocationID	zone	longitude	latitude
0	1	Newark Airport	-74.174000	40.691833
1	2	Jamaica Bay	-73.831298	40.616749
2	3	Allerton/Pelham Gardens	-73.847422	40.864474
3	4	Alphabet City	-73.976968	40.723752

4	5	Arden Heights	-74.188484	40.552660
---	---	---------------	------------	-----------

Datos meteorológicos

Se puede acceder a los datos meteorologicos con información horaria e intra horaria de la zona correspondiente NY. Para eso seleccionamos inicialmente el año de interés y descargamos el conjunto completo de datos. Luego se debe filtrar hasta obtener el conjunto de datos correspondientes a los días en los que se analizan los viajes.

```
meteo_url = (
    "https://www.ncei.noaa.gov/oa/"
    "global-historical-climatology-network/hourly/"
    "access/by-year/2024/psv/"
    "GHCNh_USW00094728_2024.psv"
)

meteo_df = pd.read_csv(
    meteo_url,
    sep="|",
    low_memory=False
)

print(meteo_df.shape)
print(meteo_df.columns.tolist())

# Se debe seleccionar un subconjunto de columnas, ya que muchos datos se presume
# que no son relevantes para el problema.
columnas_clima = [
```

```
"STATION",
"Station_name",
"DATE",
"temperature",
"dew_point_temperature",
"relative_humidity",
"wind_direction",
"wind_speed",
"wind_gust",
"precipitation",
"snow_depth",
"visibility",
"sea_level_pressure",
"sky_condition"
]

clima = meteo_df[columnas_clima].copy()
```

[Mostrar salida oculta](#)

```
# -----
# 1. Interpretar DATE como UTC
# -----

clima["DATE.UTC"] = pd.to_datetime(
    clima["DATE"],
    utc=True
)
```

```
# -----
# 2. Convertir de UTC a hora local de Nueva York
# -----

clima["DATE_NY"] = (
    clima["DATE.UTC"]
    .dt.tz_convert("America/New_York")
)
```

```
    loc_timezone = 'America/New_York',  
)
```

```
# -----  
# 3. Filtrar enero de 2024 en hora local  
# -----  
  
clima_mes = clima[  
    (clima["DATE_NY"] >= "2024-01-01 00:00:00-05:00") &  
    (clima["DATE_NY"] < "2024-02-01 00:00:00-05:00")  
].copy()  
  
print(clima_mes.shape)
```

```
(1076, 16)
```

```
# -----  
# 4. Crear una columna de hora local  
# -----  
  
clima_mes["hora"] = (  
    clima_mes["DATE_NY"]  
    .dt.floor("h")  
)
```

```
# -----  
# 5. Revisar cantidad de observaciones por hora  
# -----  
  
observaciones_por_hora = (  
    clima_mes  
    .groupby("hora")  
    .size()  
)
```

```
print(observaciones_por_hora.value_counts())
```

```
1    575
2     73
3     55
4     18
5     17
6      3
8      1
7      1
Name: count, dtype: int64
```

✓ Resumen Horario de los datos meterológicos

Los datos meteorológicos pueden tener muestras en intervalos sub horarios. Por lo que para representar el estado del tiempo meteorológico en una determinada hora hay que proceder a seleccionar una muestra representativa o agregar varias muestras para representar lo que ocurrió en esa hora en particular.

```
# -----
# 6. Variables meteorológicas numéricas
# -----
# Las variables meteorológicas numericas deben ser agregadas en los casos
# que exista más de una medicion dentro de una misma hora.

variables_numericas = [
    "temperature",
    "dew_point_temperature",
    "relative_humidity",
    "wind_speed",
    "wind_gust",
    "precipitation",
    "snow_depth",
    "visibility",
    "sea level pressure"
```

```
sea_level_pressure
```

```
# -----  
# 7. Agregar a una observación por hora  
# -----  
  
clima_horario = (  
    clima_mes  
    .groupby("hora")[variables_numericas]  
    .mean()  
    .reset_index()  
)  
  
clima_horario.head()
```

	hora	temperature	dew_point_temperature	relative_humidity	wind_speed	wind_gust	precipitation
0	2024-01-01 00:00:00-05:00	5.6	-2.8	55.0	3.1	NaN	0.0
1	2024-01-01 01:00:00-05:00	5.6	-2.2	58.0	2.1	NaN	0.0
2	2024-01-01 02:00:00-05:00	5.0	-2.2	60.0	0.0	NaN	0.0
3	2024-01-01 03:00:00-05:00	5.0	-1.7	62.0	1.5	NaN	0.0
4	2024-01-01 04:00:00-05:00	4.4	-1.1	68.0	0.0	NaN	0.0

Empieza a programar o a [crear código](#) con IA.

